MAX Jeune Planner: Architecture and Implementation Evaluation

Evaluation date: 11 October 2026. This is the companion to the code audit report. The code audit lists defects (F-01 to F-23); this report judges the framework as a whole: how good the idea is, how well its architecture carries the idea, and what it would take to mature it. Scores are my judgement, supported by the measurements and tests described in the audit. Nothing was changed in your code.

1. Verdict

The idea is strong and the implementation proves it works; the architecture has not yet caught up with the idea.

The concept (a local, account-free planner that builds multi-leg MAX Jeune itineraries from SNCF open data, with an "explore from a station" map and a date calendar) fills a real gap between the official MAX Planner and general journey planners. The domain thinking behind it is notably good: it treats the daily `OUI` flag as indicative rather than as a booking guarantee, uses SNCF's official minimum-connection-time dataset, models OUIGO Grande Vitesse separately from OUIGO Train Classique, and says plainly what it cannot know.

The implementation, however, is a very capable **prototype**: three large files with mixed responsibilities, one search approach re-implemented four times, no persistent data layer, no separation between data and presentation strings, and an English-only interface for what is mainly a French audience. Those choices explain most of the audit's High and Medium findings, so fixing structure will fix many defects at once.

Overall maturity: working prototype / late alpha. Overall score: 3.0 out of 5 (idea 4.5, execution 2.5).

2. The idea, restated

What the framework promises a user:

1. Find a way to travel between two stations using trains flagged free for MAX Jeune holders, including journeys that need several changes.
2. Show alternatives honestly: paid fallbacks, OUIGO candidates, night trains, and the closest reachable stop when no complete route exists.
3. Explore: given a start station and date, show everywhere reachable with MAX-only trains, one way, same-day return, or dated return.

4. Compare dates: show on which days a direct MAX train exists for a pair.
5. Do all of this privately on the user's own machine, without accounts or scraping the protected official planner.

What the public data can and cannot support: it gives a daily availability flag per train and station pair, a full national timetable, and a minimum-connection dataset. It does not give remaining seat counts. This sets a ceiling on how "confirmed" any result can be, and the project respects that ceiling in its wording.

3. Architecture as built

Layer	File	What it actually does	Size
Shell	`desktop/main.js`	Starts the server in-process on a random port, opens an Electron window	89 lines
HTTP + data access	`server.js`	Routing, downloading and unzipping feeds, CSV parsing, caching, input handling, orchestration, response notes	81 lines, 26,659 chars
Domain engine	`planner.js`	Station identity, availability normalisation, timetable-to-legs, connection rules, four search algorithms, sample data	301 lines, 50,538 chars
Presentation	`index.html`	Markup, six style blocks, all client logic, rendering, SVG map, booking hand-offs	165 lines, 57,369 chars
Quality aids	`test/`, `audit-*.js`	18 unit tests, three data audit scripts	298 + ~390 lines

Data flow for a journey search: browser to `GET /api/search`, then the server downloads (or reuses) MAX records and GTFS, reads the stop-times file, builds every leg for every day in range, runs a breadth-first path search twice (outbound and return), pairs same-day options, and returns JSON that already contains English warning sentences. Explore and calendar follow similar shapes with their own variants.

Structural measurements:

- `server.js` averages 329 characters per line with 14 lines over 500 characters (longest 2,109); `planner.js` has 15 lines over 500 (longest 1,882); `index.html` has 32 (longest 5,434). The test file, by contrast, averages 45 characters per line with none over 221.

The code that is tested is readable; the code that matters is not.

- `planner.js` exports 18 symbols and `server.js` imports 16 of them, so the boundary between layers is wide.
- The server keeps four module-level mutable caches (`maxCache`, `timetableCache`, `connectionCache`, `maxMetadataCache`) with no shared policy.
- There are six JSON endpoints plus the page, with no versioning or schema.

4. Scorecard

Scale: 1 = absent or harmful, 3 = adequate for a prototype, 5 = production grade.

Dimension	Score	One-line reason
Idea and product concept	4.5	Real gap, clear niche, privacy-friendly, honest about limits
Domain modelling	3.5	Right concepts (IATA identity, MCT rules, OUIGO split) but encoded as flags and strings
Search algorithm design	2.5	Works on tested cases; path enumeration with caps is slow and incomplete (F-02, F-13)
Data layer and pipeline	2.0	No refresh policy, no store, whole-range downloads, per-request recomputation (F-01, F-08, F-09)
API design	2.0	No validation, wrong status codes, display strings mixed into data (F-06, F-07)
Frontend and UX	3.0	Rich and careful copy; no localisation, race conditions, accessibility gaps (F-18)
Desktop packaging	3.5	Clean idea (same server in Electron, secure webPreferences); unsigned, macOS only (F-10, F-23)
Testing	2.0	Good regression intent; red suite, blind to the production code path (F-03, F-19)
Operability and resilience	2.0	Single-thread blocking, crash on bad request, silent truncation (F-02, F-04)
Maintainability	2.0	Dense lines, duplication, hand-kept tables (F-15, F-21)

Dimension	Score	One-line reason
Security posture	3.0	Good defaults (loopback, escaping, sandbox); gaps in Host checking and navigation guard (F-05, F-10)
Documentation and honesty	4.0	Exceptionally candid README and UI; some drift from the code (F-20)

5. What the framework gets right

These are design decisions worth keeping, because they are hard to retrofit later.

5.1 A hybrid graph: MAX records for the free network, GTFS for the rest. The availability feed defines which legs are free; the timetable supplies paid first/last legs and OUIGO candidates. Restricting paid legs to the search endpoints and admitting OUIGO Grande Vitesse between relevant nodes is a smart way to keep the graph small while still answering "what is the fastest way, free or not?".

5.2 Station identity via IATA codes with name fallback. Using the feed's own station codes avoids the classic "Paris" and "Lyon" city-versus-terminal confusion, and the explicit "Paris (all stations)" and "Lyon (all stations)" groups let users choose breadth deliberately. The `audit:stations` script, which checks every code against the timetable, is the right kind of safety net.

5.3 SNCF's official minimum-connection data instead of a guessed constant. Station-pair rules, train-specific exceptions and forbidden transfers are modelled, with a manual and a disabled mode for exploration. Most hobby planners use a flat number; this one tries to be correct.

5.4 Uncertainty is part of the product. Results are separated into MAX-first, night trains, OUIGO candidates, fastest overall and closest reachable. Brussels eligibility, OUIGO seat uncertainty and snapshot age are surfaced to the user. This is the right posture for a tool built on indicative data.

5.5 Local-first and legally cautious. No accounts, no tracking, and a deliberate decision not to automate around the official planner's bot protection. That keeps the project defensible and easy to trust.

5.6 One server, two shells. Running the same Node server in a browser (`npm start`) and inside Electron means the desktop app is a thin wrapper, not a second codebase. The Electron settings (`contextIsolation` , `sandbox` , no node integration) are correct.

5.7 Engineering aids beyond the minimum. Sample mode for offline demos, three audit scripts that check engine output against raw feed data, and regression tests written from real bugs show a team (or author) that cares about correctness.

6. Where the implementation falls short of the idea

6.1 One search idea, implemented four times

`findJourneys`, `findClosestJourneys`, `findReachableDestinations` and `findReachableDestinationOptions` each contain their own frontier loop, connection check, cap constants and path-uniqueness test. `findReachableDestinations` and `findReachableDestinationOptions` even contain a copy-pasted `findReturn`. Behaviour already differs between them: the planner uses a 30-minute same-day rule while Explore defaults to 180 minutes; the planner caps at four changes and Explore at five; caps are 60,000, 50,000, 15,000 and 12,000 in different places.

This is the single biggest structural weakness. Every bug fix (boarding flags, DST, transfer floors) must be applied four times, and users can see inconsistent answers to what is conceptually the same question.

6.2 The algorithm is the wrong class for the problem

Enumerating paths breadth-first, capped by counts, gives incomplete results and cost that grows with data volume (audit F-02, F-13). Public-transport routing has a well-established family of algorithms for exactly this: the **Connection Scan Algorithm** and **RAPTOR**. Both work directly on sorted timetable connections, handle minimum transfer times, produce earliest-arrival or Pareto-optimal sets by number of transfers, and run in milliseconds on national networks. RAPTOR's "rounds equal transfers" model matches this project's "zero to five changes" concept almost exactly, and its Pareto labels can carry the extra criteria the product cares about (number of confirmed MAX legs, paid legs, OUIGO legs).

This does not mean rewriting everything; it means replacing the four loops with one engine whose inputs are connections and rules, and whose output is a labelled set of journeys.

6.3 No real data layer

The data pipeline is: download, parse text into row objects, filter, and discard at the end of each request. Consequences measured in the audit: an 85 MB `stop_times` file is re-read per search, legs are rebuilt per request (66 s for a 30-day span), the MAX cache holds one date range, and the timetable never refreshes.

A small persistent layer would remove a whole class of problems: ingest daily into per-service-day connection tables (SQLite, or sorted binary files), record the feed version and fetch time, and have the engine read only the days it needs. Ingestion then becomes a background job with atomic swaps, and searches become reads.

6.4 Product state encoded as boolean flags

A leg's meaning is spread over `maxAvailable`, `paid`, `maxEligibilityUnknown`, `nightTrain`, plus string labels (`type`) and a free-text `trainAlternates` array. Valid combinations are implicit; for example `paid = !ouigoCandidate && !listedMaxAvailable` is derived in one place and re-derived in others. A single explicit field such as `fare: 'max_confirmed' | 'ouigo_candidate' | 'paid' | 'unknown'` would make filters, counts and UI labels straightforward and remove many conditional chains in `findJourneys`.

Station identity has the same issue: it is a mix of `stationKey` (a lossy name key), `stationIataCode`, `stationNodeKey` and `transferNodeKey`, hand-maintained in two parallel tables (F-15). A first-class `Station` object (id, display name, IATA, UIC, coordinates, group memberships) built once from the feed would replace most of the string juggling.

6.5 Presentation concerns leak across layers

- The server returns finished English sentences (`note`, `dayTripWarning`, `eligibilityWarning`) inside data responses. Clients cannot localise or restyle them.
- Business rules live in the browser as well as the server: the Brussels eligibility regex appears in the server and at least four places in `index.html`; Trainline station IDs are hard-coded in the page; a second, simplified `stationKey` exists in the client.
- Constants that define product behaviour (28-day return limit, 29-day horizon, 180/30-minute stays, 12-result limits) are scattered in both layers, and the client re-derives limits the server also enforces.

A clean contract would be: the server returns structured facts and warning codes (`eligibility: 'unconfirmed'`), the client owns wording and translation, and shared constants come from one `/api/config` response.

6.6 Language and audience fit

MAX Jeune is a French subscription for 16 to 27 year olds, station names are French, SNCF Connect and Trainline hand-offs are French, yet the interface and documentation are English only and dates and times use the browser locale. For the product's natural users this is the largest gap between idea and delivery, and it is cheap to fix early (message catalogues with French as default) and expensive to fix late (sentences are currently built by string concatenation inside server and client code).

6.7 The tool can mislead on chained journeys

A multi-change MAX journey requires every leg to be booked separately and every leg to still have a free seat. The planner shows each leg's flag and the waiting times, but not the practical risk: if the fourth leg sells out after the first three are booked, the traveller is stranded or must pay. The UI could rank or label chains by booking risk (number of legs, tightness of connections, how scarce each leg is across the 30-day snapshot) and suggest booking the scarcest leg first. This is a product feature that follows directly from the idea and is where the tool could clearly beat the official planner.

6.8 Consistency across the three modes

"Plan journey" includes paid and OUIGO legs, "Explore" is MAX-only, and "Compare dates" is direct trains only. Each has different station matching, different change limits and different notions of "available". Users cannot easily tell why Explore shows a destination that Plan does not reach, or the reverse. A shared engine (6.2) with mode-specific filters would give consistent answers and let the UI explain differences.

7. Fit against alternatives

Tool	Strength	Weakness	Where this project wins
Official MAX Planner	Live seat counts	Direct trains, no exploration, bot-protected	Multi-leg routes, explore map, no login
SNCF Connect / Trainline	Booking, live data	MAX not applied automatically; discovery is by destination, not by free availability	Free-network discovery and calendar
Hobby scripts (including the bundled third-party zip)	Automation	Fragile, terms-of-service risk	Open-data-only, defensible, local

The niche is clear: **discovery and planning across the free network**. The implementation currently serves it well for single-leg and light multi-leg searches and poorly for the heavy cases the idea most invites (long date ranges, many changes, whole-network exploration).

8. Maturity assessment

Capability	Today	Needed for a dependable release
Data freshness	Manual, one-time timetable	Daily refresh, versioned, visible age
Performance	Seconds to over a minute, blocks app	Under one second per search, off main thread
Correctness assurance	17 of 18 tests pass, engine path untested	Green suite, property tests, golden routes from real data
Observability	None	Structured logs, search diagnostics shown on request
Localisation	English only	French first, English second
Distribution	Unsigned arm64 macOS zip	Signed and notarised, universal build, update channel
Platforms	macOS desktop, Node on others	Windows and Linux builds, or a hosted web option

9. Recommended target architecture

Concern	Today	Proposed
Ingestion	Inline in request handlers	Separate module: fetch, validate, version, atomic swap, scheduled
Storage	Row objects in memory per request	SQLite or per-day connection files, indexed by day and station
Domain model	Plain objects with flags	Typed `Station`, `Connection`, `Journey` with an explicit fare class
Engine	Four BFS loops	One RAPTOR or CSA engine, pure functions, worker thread
Rules	Mixed in planner	Connection-rules module with its own tests
API	Unvalidated query strings, English prose	Schema-validated requests, versioned JSON, warning codes
Frontend	One 57 KB file	Components or modules, message catalogue (fr, en), shared constants

Concern	Today	Proposed
Quality	One test file	Unit tests per module, API tests, fixture-based route tests, CI
Shell	Server in main process	Engine in a utility process; Electron only renders

TypeScript (or JSDoc types checked by `tsc`) would pay off immediately given how many shapes (legs, journeys, rules, stations) flow between files.

10. Suggested roadmap

1. **Stabilise (about 1 week).** Apply the audit's safety fixes (validation, error handling, Host check), fix the failing test, add the timetable refresh and atomic downloads, and format the code with Prettier. No behaviour change beyond bug fixes.
2. **Unify (2 to 3 weeks).** Introduce the `Station` and `Connection` types, a single fare-class field, and one shared search routine used by plan, closest-stop and explore. Move constants to one config module. Add tests for the SNCF rules path.
3. **Accelerate (2 to 4 weeks).** Build the per-day connection store and replace path enumeration with RAPTOR or CSA behind the same function signatures. Compare outputs against the old engine on recorded searches before switching.
4. **Localise and clarify (1 to 2 weeks).** Return warning codes from the API, add a French message catalogue, add the chained-journey booking-risk indicator.
5. **Ship (1 to 2 weeks).** Code-sign and notarise, produce a universal macOS build, decide on Windows and Linux, and add CI with test, audit-script and `npm audit` gates.

The order matters: steps 1 and 2 reduce risk and make step 3 safe, and step 3 is where most users will notice the improvement.

11. Limitations of this evaluation

Scores are qualitative judgements, not benchmark results. I did not test the live SNCF services, the Electron GUI on a Mac, or real user behaviour, and I did not interview the author about intent, so some choices that look accidental may be deliberate. The RAPTOR and CSA recommendation is based on their documented properties; a prototype on this data would confirm the speed-up before committing. Legal points (open-data licence attribution, the third-party code in `sncf-max-main.zip`) should be checked against the actual licence texts before distribution.